

Stockbot / Capital Ledger — Architecture

A single-user automated equity trading system for acting on SPTulsian advisory calls.

Version as of 2026-08-16 · Ubuntu 22.04 on OCI · Python 3.10 · Oracle Autonomous Database

1. What this system is

SPTulsian is a paid Indian equity advisory. It emails stock recommendations on market mornings and publishes targets on a member portal. Acting on those calls by hand means reading email at 9am, deciding position size against a budget policy, placing buy orders, then remembering to place a sell trigger at the advisory's target — for every open position, every day.

This system does that work unattended, and keeps a ledger of what it did.

It is deliberately **not** a trading strategy. It does not decide *what* to buy; SPTulsian does. The system decides *how much*, subject to a written budget policy, executes reliably, and reports honestly on the outcome. A separate conviction layer assesses how well each recommendation is supported by public evidence, but that layer is advisory only and cannot affect an order.

Design priorities, in order

1. **Never place a wrong trade.** Ambiguity halts and asks a human.
2. **Never lose the ledger.** What happened must be recoverable and truthful.
3. **Fail loudly.** Silence must not be mistakable for a quiet day.
4. **Then, automate.**

Priority 1 is why the buy flow refuses to guess a ticker symbol. Priority 3 is why a scraper outage raises an alarm rather than simply producing no targets.

2. Functional view

2.1 The daily cycle

09:30 IST	RECOMMEND	Read overnight advisory email. Resolve each tip to a Kite trading symbol. Scrape the portal for target / timeframe. Write PENDING_BUY (clean) or NEEDS_REVIEW (ambiguous). No money moves. 90-minute review window
10:15 IST	CONVICTION	Score each new recommendation on public evidence. Display only – cannot block or size a buy.
10:45 IST	WATCHDOG	Confirm the scraper actually ran. Email if not.
11:00 IST	BUY	For each PENDING_BUY: price it, size it against the budget policy, place a real order. Re-attempt any NEEDS_REVIEW fixed in the window.
16:00 IST	GTT	For filled buys with a target: place a GTT sell. For triggered GTTs: confirm the sell actually filled, then close the trade and recycle its budget.

The gap between 09:30 and 11:00 is the most important design decision in the system. Symbol resolution is where a catastrophic error lives — buying ICDSLTD (a ₹52 crore shell) when the advisory said CDSL (a ₹28,000 crore depository) is a real incident this system has had. Splitting recommendation from purchase creates a window in which a human can correct a bad symbol *before* money moves, rather than discovering it afterwards.

2.2 Position sizing

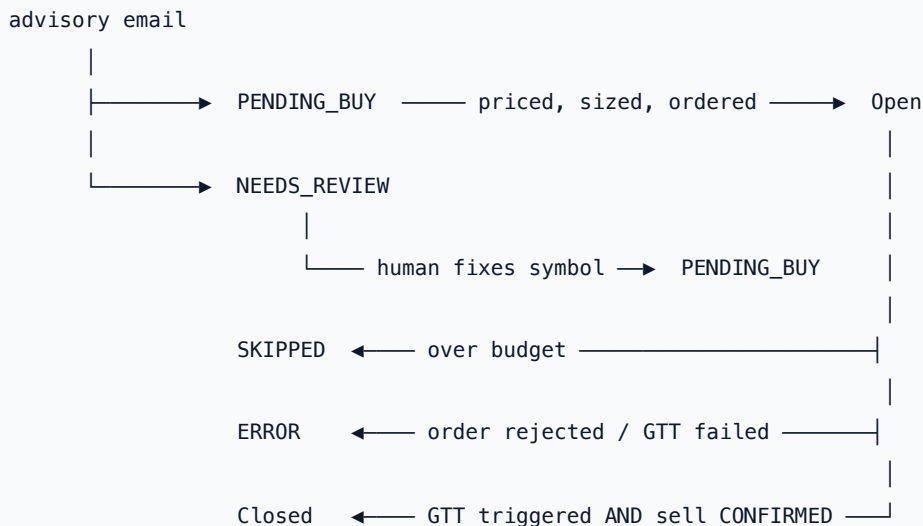
Each buy is capped by two independent rules, both read from the database:

Rule	Meaning
Category allocation	Each advisory section (Big Gems, Little Gems, ...) gets a percentage of total portfolio budget.
Per-stock cap	A single stock may not exceed a percentage of total budget, by market-cap class: Large 10%, Mid 6%, Small 4%, Micro 2%.

The per-stock cap is **per individual stock**, not a shared pool across all stocks of that cap class. This distinction was originally implemented wrongly — as a shared bucket — which silently skipped legitimate trades once a few small-caps had been bought. Market-cap class comes from AMFI's official half-yearly categorisation, held in STOCK_CAP_CLASSIFICATION .

Budget is recycled when a position closes, so the ledger reflects capital actually at risk rather than cumulative spend.

2.3 Trade lifecycle



Closed requires a **confirmed fill**, not merely a triggered GTT. A GTT can leave the active state three ways: it triggered and the sell filled (a real close), it triggered but the day-validity sell order expired unfilled, or it was cancelled. Only the first closes the trade; the other two get a fresh GTT at the same target, tagged with a retry counter, so a position is never left silently unprotected.

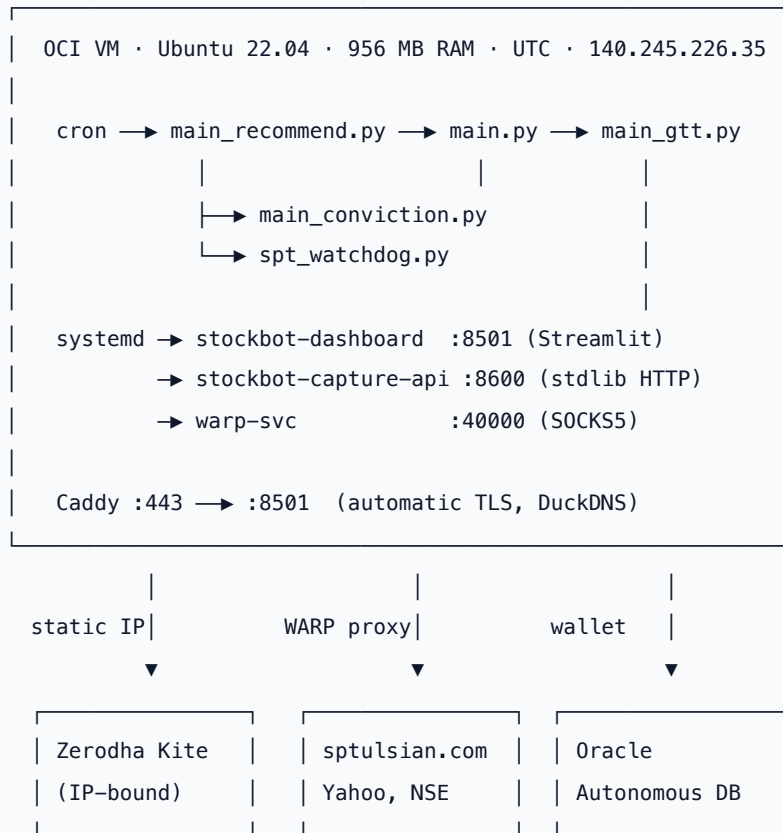
2.4 The dashboard

A Streamlit application at <https://raramdas-stockbot.duckdns.org>, usable from a phone via Add to Home Screen. Login is a password with an encrypted 30-day "remember me" cookie.

Page	Purpose
Overview	Budget deployed, live holdings, current value
Category Drill-Down	Holdings and P&L within one advisory section
Performance	Realised/unrealised P&L, cumulative chart, win rate
Set Targets	Edit target price / timeframe per open trade
Trades Explorer	Filterable full trade history, CSV export
Recommendations	Every tip ever seen, bought or not
Conviction	Evidence-based score per position, with full working
Needs Review	Fix a bad symbol, preview, then confirm a real buy
Open Orders	Live Kite orders and GTTs
Classification	AMFI market-cap lookup
Settings & Edits	Budget, allocations, manual trade close

3. Technical view

3.1 Topology



Two deployment directories are kept in sync from one Git repository: `/home/ubuntu/stock_bot_v4` (cron jobs) and `/home/ubuntu/stockbot` (dashboard).

3.2 Egress split — the load-bearing constraint

`sptulsian.com` sits behind CloudFront, whose WAF rejects datacenter IP ranges **as a class**. The VM received HTTP 403 with CloudFront error headers on every path, including unauthenticated ones — the origin never saw the request. This is not rate limiting and not bot detection: no user-agent, header set, cookie jar or request pacing changes the outcome, because the discriminator is the source address.

Cloudflare WARP, run in **proxy mode**, egresses from consumer-registered addresses that the WAF serves normally. `warp-svc` exposes SOCKS5 on `127.0.0.1:40000`.

The critical property is **scope**. Zerodha's API is bound to this VM's registered static IP; a system-wide VPN or firewall redirect would move order placement onto Cloudflare's egress and break that binding. So the proxy is attached to exactly one object — the scraper's `requests.Session` — and nothing else:

```
# spt_scraper.build_session()
proxy = os.environ.get('SPTULSIAN_PROXY', '') # socks5h://127.0.0.1:40000
if proxy:
    session.proxies = {'http': proxy, 'https': proxy}
```

Two details carry real weight:

- The variable is named `SPTULSIAN_PROXY`, **not** `HTTP_PROXY` / `HTTPS_PROXY`. `requests` honours the standard names automatically for every module in the process, which would silently route broker traffic through the proxy.
- The scheme is `socks5h`, not `socks5`. The trailing `h` resolves DNS at the proxy, so hostname lookups also traverse Cloudflare rather than leaking from the blocked address.

The one check that matters. If line 1 ever returns a Cloudflare address, the proxy has escaped its scope and broker traffic is at risk:

```
curl -s ifconfig.me # → 140.245.226.35
curl -s --socks5-hostname 127.0.0.1:40000 ifconfig.me # → a Cloudflare address
```

Traffic	Egress	Why
Zerodha orders, holdings, GTTs	Static IP	Broker IP binding — must never move
Gmail IMAP, Oracle	Static IP	No reason to proxy
sptulsian.com, Yahoo, NSE	WARP SOCKS5	The only traffic that is blocked

3.3 Advisory portal scraping

The portal serves its sections through **two different backend controllers**, which requires two parsing strategies:

- **JSON** — the page wires up a per-section endpoint (`/getMWActiveData` for Little Gems, `/getFCActiveData` for Big Gems). The endpoint name is read off each page rather than hardcoded: calling one section's endpoint for another returns HTTP 500, which is easily misdiagnosed as a portal outage.
- **HTML** — the remaining sections ship no data endpoint and render calls directly into the page markup. Row markup is not positionally stable (the same table emits 11- and 13-cell rows, carrying both desktop and mobile cells), so fields are read from row text by content rather than column index.

Three correctness rules govern what is done with a scraped call:

Login is verified on substance, not markers. Checking for a "logout" link passes while signed out — the public teaser page contains that string in its navigation. A session is accepted only if it yields parsed call rows, which no signed-out response can produce.

Archived is not the same as closed. Little Gems and Big Gems carry no active list on this subscription, so even same-day calls appear under "Archives". Closure is determined by an actual exit remark (`Target met...`), not by which table a row sits in. Treating archived as closed would freeze out 82 of 86 open positions.

Calls are matched within their own section. The same stock is called in several sections with different targets and horizons — a Multibagger call may target 2027 at a far higher price than a 3-month Big Gems call on the same stock. Applying the wrong section's target arms a GTT that never triggers and parks the capital.

Regular Income Bluechips is excluded entirely: it is a covered-call section whose "Target" is a total position value (e.g. ₹7,80,000), not a per-share price.

3.4 Conviction engine

Answers a question the advisory does not: *is this call well-supported by public evidence?* Four weighted layers over a 100-point composite.

Layer	Points	Checks
Fundamentals	40	Piotroski F-Score, Altman Z''-EM, Beneish M-Score, ROE, debt/equity, FCF yield, revenue growth
Consensus	25	Analyst rating, target upside, coverage breadth, advisory target sanity
Technical	20	50/200-DMA alignment, RSI overbought guard, 52-week exhaustion, volume confirmation, liquidity
Governance	15	NSE ASM/GSM surveillance, promoter holding, institutional holding, results event risk

Sources: Yahoo Finance (fundamentals, consensus, price history), NSE (surveillance). Both are fetched through the WARP proxy and both fail soft.

Three departures from the conventional design, each driven by what this portfolio actually contains:

Missing data renormalises; it does not derate. The portfolio is small- and micro-cap heavy, and 13 of 37 held symbols have zero analyst coverage. Deducting points for absent coverage would systematically penalise the entire Little Gems category — encoding "*we know less*" as "*this is worse*". Instead the score reports **quality** on what was measurable, and a separate **evidence** figure reports how much of the 100-point frame could be assessed. Layers enter the composite weighted by coverage, so a layer resting on one surviving check carries proportionally less weight rather than extrapolating that check across its whole budget.

UNKNOWN (tried and failed) reduces evidence. **N/A** (the metric never applied) does not.

Financials skip the accrual models. Piotroski, Altman and Beneish assume an operating balance sheet; current ratio, gross margin and asset turnover do not transfer to a lender. Financial Services names mark those checks N/A and renormalise rather than producing confident nonsense.

Gates are separated from warnings. A gate forces recommend-reject; a warning is surfaced and leaves the verdict to the score. Tuned against the live portfolio, where over-eager gates produced obvious false positives:

- **Beneish is a warning, never a gate.** Its sales-growth term over-flags fast growers; on this portfolio it flagged BEL, CG Power, Solar Industries and Mazagon Dock. A gate that rejects those trains the reader to ignore the tool.
- **Altman gates only below 3.0**, not at the 4.15 grey line. Vodafone Idea reads 0.57 and is unambiguous; capital-intensive manufacturers sit under 4.15 structurally.
- **Liquidity and GSM surveillance remain gates.** ₹0.00 crore/day of traded value means a GTT may never fill, regardless of how good the company is.

Below the evidence floor (40 of 100 points) no composite is published at all — a delisted symbol scoring "100/100" off one surviving governance check is worse than reporting nothing.

The engine is display-only. `main_conviction.py` writes to `CONVICTION_SCORES` and nothing else. It cannot size a position or stop a buy. This is deliberate: the scores must be checkable against known outcomes before anyone considers wiring them into sizing.

Validation: backfilled across all 86 open positions, it independently flagged both trades already known to be mistakes — `ICDSLTD` (illiquid, ₹0.00 crore/day) and `NIFTY INFRA` (an index, insufficient evidence) — without being told about either.

3.5 Component reference

Module	Role
<code>config.py</code>	Env/credentials, <code>KITE_ACCOUNT</code> switch (NEW/OLD), logging, IST
<code>email_reader.py</code>	Gmail IMAP; parses the advisory email into tips
<code>kite_client.py</code>	Kite login (TOTP), symbol resolution, order and GTT placement
<code>order_status.py</code>	Order status lookup and sell-order reconciliation
<code>budget_manager.py</code>	Budget policy, all <code>TRADES</code> reads/writes
<code>spt_scraper.py</code>	Portal login, both parsing strategies, liveness watermark
<code>conviction.py</code>	Evidence gathering and the four scoring layers
<code>main_recommend.py</code>	Phase 1 — resolve symbols, scrape targets, no orders
<code>main.py</code>	Phase 2 — price, size, buy
<code>main_gtt_oracle.py</code>	Phase 3 — place GTTs, confirm fills, close trades
<code>main_conviction.py</code>	Score today's recommendations (display only)
<code>spt_watchdog.py</code>	Liveness alarm for the scraper
<code>spt_capture.py</code>	Manual review-then-save of scraped targets
<code>dashboard/app.py</code>	Streamlit UI, all pages
<code>dashboard/db.py</code>	Dashboard's Oracle data access
<code>dashboard/kite_data.py</code>	Multi-account Kite sync, retry-buy preview/confirm
<code>dashboard/capture_api.py</code>	Authenticated endpoint for <code>spt_capture.py</code>
<code>dashboard/theme.py</code>	CSS design system, table and KPI rendering

3.6 Data model

Oracle Autonomous Database, wallet authentication.

Table	Contents
TRADES	The ledger. One row per recommendation, whatever its outcome.
PORTFOLIO_BUDGET	Total capital under management
CATEGORY_ALLOCATION	Per-section allocation % and per-cap-class caps
STOCK_CAP_CLASSIFICATION	AMFI market-cap categorisation
CONVICTION_SCORES	Score history; full working stored as JSON
KITE_HOLDINGS_SNAPSHOT	Holdings per account (<code>account_label</code> NEW/OLD)
KITE_GTT_SNAPSHOT	Live GTTs per account
KITE_ORDERS_SNAPSHOT	Recent orders per account
KITE_SYNC_LOG	Per-account last-synced timestamps

TRADES is the single source of truth. Every recommendation writes a row regardless of outcome, which is what makes the Recommendations page a complete record rather than a survivor-biased one.

3.7 Schedule

The VM clock is **UTC**; IST is UTC+5:30.

Job	Cron (UTC)	IST	Purpose
main_recommend.py	0 4 * * 1-5	09:30	Phase 1
main_conviction.py	<i>(not yet scheduled)</i>	~10:15	Scoring
spt_watchdog.py	15 5 * * 1-5	10:45	Liveness alarm
main.py	30 5 * * 1-5	11:00	Phase 2
main_gtt_oracle.py	30 10 * * 1-5	16:00	Phase 3
DuckDNS refresh	* / 5 * * * *	—	Dynamic DNS

4. Failure handling

4.1 The silence problem

A dead scraper and a quiet trading day look identical from outside. The buy flow deliberately continues when a scrape fails — recording the recommendation and resolving its symbol matter more than the target, which can be filled in later — so nothing in the pipeline raises. Without a dedicated check, the system could run blind for days while every log line still read "complete".

The fix is a **liveness watermark**: a timestamp written only after a section was genuinely fetched *and* parsed, never on a mere attempt. It is a signal a failed fetch cannot fake. Judging freshness from the newest row in TRADES was rejected — a genuinely quiet week would then raise a false alarm.

`spt_watchdog.py` applies two independent rules and emails on either:

Rule	Condition	Rationale
Absolute	Watermark older than 30 h	Backstop for outages spanning a day
Trading-day	Past 10:30 IST on a weekday, watermark must carry today's date	The absolute rule alone would not fire until the next evening — after a whole session had run blind

The scrape runs *before* the no-tips early return, making the watermark a true daily heartbeat. Otherwise a market holiday — a weekday with no tips — would be indistinguishable from an outage and would cry wolf every time.

4.2 Diagnostic matrix

Symptom	Likely cause	Check
Connection refused to <code>127.0.0.1:40000</code>	WARP down	<code>sudo systemctl status warp-svc ; warp-cli status</code>
HTTP 403, CloudFront headers	Egress blocked or proxy bypassed	Compare direct vs proxied <code>ifconfig.me</code>
HTTP 500 from a section endpoint	Wrong controller for that section	Endpoint is read per page — confirm the page still declares it
Pages fetch, zero rows	Portal markup changed	<code>python3 spt_capture.py --dry-run</code>
Login "succeeds", no data	Credentials changed	Substance check should catch it; verify <code>SPT_USERNAME / SPT_PASSWORD</code>

4.3 Safety invariants

These hold by construction and should be preserved by any future change:

1. Broker traffic egresses from the registered static IP. Always.
2. A symbol that does not resolve cleanly is never bought — it becomes `NEEDS_REVIEW` and waits for a human.
3. A trade is `Closed` only on a **confirmed** sell fill, never on a GTT status alone.
4. Conviction scores cannot size, gate, or block an order.
5. Manual buy paths (Needs Review, `spt_capture`) always preview before they commit, and the confirm step is the only thing that spends money.

5. Operations

5.1 Deploy

```
# local
git add -A && git commit && git push

# VM – both directories track the same repo
ssh -i ~/.ssh/kite_key ubuntu@140.245.226.35 '
  cd /home/ubuntu/stock_bot_v4 && git pull -q
  cd /home/ubuntu/stockbot      && git pull -q
  sudo systemctl restart stockbot-dashboard'
```

5.2 Health check

```
ssh -i ~/.ssh/kite_key ubuntu@140.245.226.35 '
  curl -s ifconfig.me; echo                # must be 140.245.226.35
  curl -s --socks5-hostname 127.0.0.1:40000 ifconfig.me; echo # must be Cloudflare
  sudo systemctl is-active warp-svc stockbot-dashboard stockbot-capture-api
  cd /home/ubuntu/stock_bot_v4 && python3 spt_watchdog.py --check-only'
```

5.3 Testing convention

Anything that touches money or the ledger is exercised in an isolated directory with Oracle writes, Sheet writes, and order placement monkey-patched out, run against **real live data**, and inspected before the real run. Mocked-write test harnesses have caught, among others: the pooled-budget bug, the GTT close that recorded no sell price, and a `DataFrame.__bool__` call that silently nulled every financial statement in the conviction engine.

Automated trading agents do not place real trades on the operator's behalf during development. Commands that move money are handed to the operator to run.

6. Known limitations

- **WARP is a single point of failure** for the advisory feed, and a consumer service with no availability guarantee. Mitigated, not eliminated: the email channel still delivers call headlines, the watchdog surfaces an outage within one trading morning, and `spt_capture.py` remains a manual break-glass path.
- **Yahoo Finance is an unofficial source** for fundamentals. Coverage is good for this portfolio (all 37 held symbols return 4–5 years of statements) but is neither contractual nor audited.
- **Conviction governance is partial.** Promoter *pledge* and holding *trend* need shareholding-pattern filings, which are not yet parsed; only point-in-time holding percentages are scored.
- **Conviction is unscheduled.** It runs on demand until it has a track record worth trusting.

- **Analyst consensus is thin for micro caps** by nature. Handled by renormalisation, but it means the Consensus layer contributes little for much of Little Gems.
 - **Single tenant in practice.** `TENANT_CONFIG` and the `bot/` tree carry multi-tenant scaffolding that is not currently scheduled.
-

7. Note on the conviction layer

The conviction engine computes published metrics from public data and shows its working. It is decision support, not financial advice, and not a prediction. Every threshold in it is a convention rather than a fact, tuned against one portfolio. The judgement stays with the human reading the output — which is precisely why the layer is display-only and why every score is shown alongside the checks that produced it.